CAHIER DU PARTICIPANT · SYNTHÈSE ET AIDE-MÉMOIRE

Introduction à Python

Résumé théorique, aide-mémoire syntaxique C# / Java / JS vers Python et exercices pratiques

1. Portrait de Python : philosophie, forces et limites

Créé par **Guido van Rossum** (1991) et nommé en clin d'œil à la troupe humoristique *Monty Python*, Python a été conçu autour d'un principe directeur : **la lisibilité du code**.

✅ Pourquoi choisir Python ? (Forces)

- **Productivité élevée :** Syntaxe concise (3 à 5x moins de lignes qu'en Java/C++).
- **Piles incluses (*batteries included*) :** Bibliothèque standard très riche dès l'installation.
- **Écosystème géant (PyPI) :** Plus de 550 000 paquets gratuits.
- **Langage « couteau suisse » :** Numéro 1 en IA, science des données, automatisation TI et scripts DevOps.

⚠️ Quand faire attention ? (Limites)

- **Vitesse d'exécution :** Langage compilé en bytecode et exécuté sur la machine virtuelle CPython (plus lent que C++/Rust/C# sur les boucles pures).
- **GIL (*Global Interpreter Lock*) :** Limite le parallélisme CPU multi-fils (*multithreading*) au sein d'un seul processus.
- **Moins adapté pour :** Applications mobiles natives, moteurs de jeux 3D lourds ou petits exécutables autonomes.

2. Aide-mémoire : passer de C#, Java ou JavaScript à Python

Concept	C# / Java / JavaScript	Python 3 🐍
Blocs de code	Accolades `{ ... }` et point-virgule `;`	Deux-points `:` suivi d'une **indentation de 4 espaces**
Style de nommage	`camelCase` (ex. `calculerTotal`)	`snake_case` (ex. `calculer_total`)
Commentaires	`// commentaire`	`# commentaire` (ou `"""docstring"""`)
Valeur nulle / booléens	`null`, `true`, `false`	`None`, `True`, `False` (attention à la majuscule !)
Opérateurs logiques	`&&` (ET), `||` (OU), `!` (NON)	`and`, `or`, `not` (en toutes lettres)
Interpolation de texte	`$"Bonjour {nom}"` (C#) / `` `Bonjour ${nom}` `` (JS)	`f"Bonjour {nom}"` (chaîne formatée / *f-string*)
Conditions multiples	`if (...) { } else if (...) { } else { }`	`if ...:` / `elif ...:` / `else:`
Longueur d'une collection	`liste.Count` / `liste.length`	`len(liste)` (fonction globale)

3. Syntaxe de base, types et structures de contrôle

Variables, conversions et opérateurs

```
# Typage dynamique mais FORT
hote = "srv-mtl-01"       # str
port = int("8080")        # conversion str ->
int
taux = float("0.14975")   # conversion str ->
float
actif = True              # bool

# Opérateurs arithmétiques particuliers
div_reelle = 17 / 5       # 3.4 (toujours
float)
div_entiere = 17 // 5     # 3 (quotient
entier)
reste = 17 % 5            # 2 (modulo)
puissance = 2 ** 10       # 1024 (2 exposant
10)

# Chaîne formatée (f-string) avec arrondi :
print(f"Serveur {hote}:{port} (taux:
{taux:.2f})")
```

Conditions et boucles

```
# Condition avec comparaison chaînée
if 200 <= code_http < 300:
    print("Succès")
elif code_http == 404:
    print("Introuvable")
else:
    print("Erreur")

# Boucle for sur une liste
serveurs = ["web-01", "db-01"]
for srv in serveurs:
    print(srv)

# Avec l'indice (enumerate) ou un compteur
(range)
for i, srv in enumerate(serveurs, start=1):
    print(f"#{i} : {srv}")
```

4. Les 4 collections fondamentales en Python

Structure	Création	Modifiable ?	Opérations courantes et cas d'usage
Liste (`list`)	`ports = [80, 443]`	Oui	`ports.append(8080)` (ajouter à la fin) `ports[0]` (premier), `ports[-1]` (dernier) Tri : `ports.sort()` ou `sorted(ports)`
Dictionnaire (`dict`)	`u = {"id": 1, "role": "ti"}`	Oui	Accès clé-valeur en *O*(1) : `u["id"]` Accès sécurisé : `u.get("courriel", "aucun")` Parcours : `for cle, val in u.items():`
Tuple (`tuple`)	`coord = (45.50, -73.56)`	**Non** (immuable)	Idéal pour des valeurs fixes ou retourner plusieurs résultats d'une fonction. Déballage : `lat, lon = coord`
Ensemble (`set`)	`tags = {"web", "prod"}`	Oui	**Élimine automatiquement les doublons** : `set(ma_liste)`. Recherche ultra-rapide : `"prod" in tags`

5. Fonctions et idiomes « pythoniques » (*Pythonic*)

1. Découpage (*slicing*) `[début:fin:pas]`

```
lettres = ["a", "b", "c", "d", "e"]
lettres[1:4]    # ["b", "c", "d"] (fin exclue)
lettres[:2]     # ["a", "b"] (les 2 premiers)
lettres[-2:]    # ["d", "e"] (les 2 derniers)
lettres[::-1]   # ["e", "d", "c", "b", "a"] (inversé)
```

2. Compréhensions de liste

```
nombres = [10, 25, 80, 150, 30]
# Filtrer et transformer en une seule ligne :
grands = [n * 2 for n in nombres if n >= 50]
# Résultat : [160, 300]
```

3. Fonctions avec annotations et retour multiple

```
def analyser_valeurs(mesures: list, seuil: int = 100) -> tuple:
    """Retourne (min, max, nb_alertes)."""
    nb_alertes = sum(1 for m in mesures if m > seuil)
    return min(mesures), max(mesures), nb_alertes

# Appel avec argument nommé et déballage :
mini, maxi, alertes = analyser_valeurs([20, 140, 85], seuil=120)
```

4. Gestionnaire de contexte `with`

```
# Ferme automatiquement le fichier même en cas d'erreur :
with open("rapport.txt", "w", encoding="utf-8") as fichier:
    fichier.write("Opération terminée.\n")
```

6. Panorama de l'écosystème et exercices de l'atelier

📦 Les bibliothèques à connaître

- **Web / API :** `FastAPI`, `Django`, `Flask`
- **Données :** `Pandas`, `Polars`, `NumPy`, `Matplotlib`
- **IA / ML :** `Scikit-learn`, `PyTorch`
- **TI / Tests :** `Requests`, `Pytest`, `Playwright`, `uv`, `ruff`

💻 Continuer sur votre machine après le cours

1. Installer **Python 3.12+** (`python.org`) ou `uv`.
2. Créer un environnement virtuel isolé par projet : `python -m venv .venv`
3. Installer des paquets sans polluer le système : `pip install pandas requests`

🧪 Exemples intégrateurs (dans Google Colab)

- **Exemple 1 (Partie 1) — Analyseur de journaux serveur :** Découper des événements texte avec `.split("|")`, déballer les variables, compter les erreurs `"ERROR"` et extraire les latences moyenne et maximale.
- **Exemple 2 (Partie 2) — Gestionnaire de billets de support TI :** Extraire l'ensemble (`set`) des catégories uniques, filtrer les urgences non résolues avec une compréhension de liste, et produire un bilan statistique via une fonction.
- **Partie 3 — Démonstrations :** Générateur de mot de passe sécurisé (`secrets`), météo en direct (API REST JSON), analyse de données et graphique instantané avec `pandas`.

Notes personnelles :